PART 1: ARTIFICIAL INTELLIGENCE

Assignment 1: Uninformed Search (DFS & BFS)

Title: Implementation of Depth First Search (DFS) and Breadth First Search (BFS) algorithms using an undirected graph.

Aim: To develop a recursive and iterative algorithm for searching all vertices of a graph or tree data structure.

Theory: Uninformed search, also known as blind search, operates without any domain-specific knowledge. It only knows how to traverse the tree and identify the goal.

- **Breadth First Search (BFS):** This algorithm starts at the root node and explores all neighboring nodes at the present depth level before moving on to nodes at the next depth level. It uses a **Queue** (First-In-First-Out) data structure. It is optimal for finding the shallowest goal.
- **Depth First Search (DFS):** This algorithm starts at the root and explores as far as possible along each branch before backtracking. It uses a **Stack** (Last-In-First-Out) or recursion. It is memory efficient compared to BFS but may not find the shortest path.

Algorithm (BFS):

1. Initialize an empty Queue and a list for Visited nodes.
2. Enqueue the starting node and mark it as Visited.
3. While the Queue is not empty: a. Dequeue node n. b. Process/print node n. c. For each neighbor m of n that is NOT visited: i. Mark m as Visited. ii. Enqueue m.

Algorithm (DFS):

1. Initialize a Stack and a list for Visited nodes.
2. Push the starting node onto the Stack.
3. While the Stack is not empty: a. Pop node n from the Stack. b. If n is not in Visited: i. Mark n as Visited and process it. ii. Push all unvisited neighbors of n onto the Stack.

Assignment 2: Informed Search (A* Algorithm)

Title: Implementation of A* Algorithm for the 8-puzzle game.

Aim: To understand heuristic-based informed search strategies and apply them to solve the 8-puzzle problem.

Theory: A* Search is one of the most popular techniques used in pathfinding and graph traversals. Unlike BFS/DFS, it uses a heuristic function to guide the search. The efficiency of A* is defined by the formula: **f(n) = g(n) + h(n)**

- **g(n):** The actual cost to reach node n from the start state.
- **h(n):** The estimated (heuristic) cost from node n to the goal. In an 8-puzzle, this is usually the number of misplaced tiles or the Manhattan distance.
- **f(n):** The total estimated cost of the lowest-cost solution through node n.

Algorithm:

1. Place the starting node in the OPEN list (priority queue).
2. While OPEN is not empty: a. Select the node with the lowest f(n) from OPEN. Let this be current. b. If current is the goal state, stop and return the path. c. Move current from OPEN to the CLOSED list. d. For each neighbor/successor of current: i. If neighbor is in CLOSED, ignore it. ii. Calculate g, h, and f for the neighbor. iii. If neighbor is not in OPEN OR the new g value is lower than its previous g value: - Update the neighbor's f value. - Set the parent of neighbor to current. - Add neighbor to OPEN.

Assignment 3: Greedy Search Algorithm

Title: Implementation of Greedy Search for Selection Sort or Minimum Spanning Tree.

Aim: To implement a Greedy Search algorithm that makes locally optimal choices to find a solution.

Theory: A Greedy algorithm builds up a solution piece by piece, always choosing the next piece that offers the most obvious and immediate benefit.

- **Selection Sort:** This is a greedy approach to sorting. It repeatedly finds the minimum element from the unsorted part and puts it at the beginning.
- **Kruskal's Algorithm:** A greedy algorithm used to find the Minimum Spanning Tree (MST) for a connected weighted graph. It sorts all edges by weight and adds the smallest edge that doesn't form a cycle.

Algorithm (Selection Sort):

1. Start with the first element of the array.
2. Search the entire array to find the smallest element.
3. Swap the smallest element with the element at the current position.

4. Move to the next position and repeat until the array is sorted.

Assignment 4: Constraint Satisfaction Problems (CSP)

Title: Solving N-Queens or Graph Coloring problem using Backtracking.

Aim: To place N queens on an NxN board such that no two queens attack each other.

Theory: A Constraint Satisfaction Problem consists of:

1. **Variables (X):** The positions to be filled.
2. **Domains (D):** The possible values (rows/columns).
3. **Constraints (C):** The rules (no two queens in the same row, column, or diagonal). **Backtracking** is used to solve this by trying to build a solution incrementally. If a constraint is violated, the algorithm "backtracks" to the previous step and tries a different value.

Algorithm (N-Queens):

1. Start in the leftmost column.
2. If all queens are placed (column > N), return true.
3. For each row in the current column: a. Check if it is "Safe" to place a queen (check left row and diagonals). b. If Safe: i. Mark this cell [row, col] as a Queen. ii. Recursively move to the next column. iii. If the recursive call returns true, the problem is solved. iv. If the call returns false, unmark the cell (Backtrack).
4. If no row works, return false.

PART 2: CLOUD COMPUTING

Assignment 1: Amazon EC2 Instance

Title: Case study and configuration of Amazon EC2.

Aim: To learn how to provision a virtual server in the cloud.

Theory: Amazon Elastic Compute Cloud (EC2) provides resizable compute capacity. It reduces the time required to obtain and boot new server instances to minutes.

- **AMI (Amazon Machine Image):** A template that contains the software configuration (OS, application server).
- **Instance Types:** Different combinations of CPU, memory, and storage (e.g., t2.micro).

Steps:

1. Log in to the AWS Console.
2. Navigate to the EC2 Dashboard and click **Launch Instance**.
3. Choose an **AMI** (e.g., Amazon Linux 2).
4. Select an **Instance Type** (t2.micro for Free Tier).
5. Configure **Security Groups** to allow SSH (port 22) and HTTP (port 80).
6. Create/Select a **Key Pair** for authentication.
7. Launch and connect via SSH.

Assignment 3: Salesforce Apex Application

Title: Creating a Salesforce application using Apex.

Aim: To develop custom logic using Salesforce's proprietary language.

Theory: Apex is an object-oriented, strongly typed programming language used on the Force.com platform. It allows developers to add business logic to system events like button clicks or record updates. It is similar to Java but specifically designed for data manipulation in the Salesforce database.

Steps:

1. Create a **Salesforce Developer Edition** account.
2. Open the **Developer Console**.
3. Go to File > New > Apex Class.
4. Write the logic (e.g., a simple discount calculator for Opportunity records).
5. Use **SOQL** (Salesforce Object Query Language) to retrieve data.
6. Save and execute the code in the "Anonymous Window" for testing.